

Microduck Studio: Four-Scenario Verification

George Hu

September 8, 2026

Scope and result

This update repairs the existing single-page Studio rather than replacing the training stack. The four scenarios are **Dance imitation, Self-pumped swing, Fast running, and Stilt walking**. The previously trained final policies were retained and independently evaluated again. No skill thresholds were relaxed.

All four policies passed all five fresh deterministic audit rollouts, and their saved API evaluations each report 16/16 passing episodes. The fresh audit also checks negative controls, the 61-observation/14-action ONNX contract, numerical parity, and motion videos. These are **nominal MuJoCo simulation results**, not physical robot tests. The five seeds are resets of each selected actor, not five independently trained models.

The only new training in this UI-fix task is an explicitly separate four-transition Cumbia clip smoke run. It proves the selected input reaches training, evaluation, rendering, and export. It is not claimed to acquire the Cumbia skill.

What changed and why

Reference selection and exact settings

The Dance selector discovers valid JSON clips from `dance-clip/` and selected extracted references under `rlx/artifacts/`. It shows the name and duration; malformed clips, invalid joint/time data, and escaping symlinks are excluded. The chosen path is submitted as `danceClip`, not merely played as an animation. In Full mode, the episode and video horizons become the clip duration and the evaluation horizon becomes its duration times 50 control steps per second, rounded upward. A new clip still needs training and independent skill evaluation.

Saved-run selection restores the complete saved evaluation recipe, including rollout size, minibatches, episode horizon, pose tolerance, fixed locomotion command, reward weights, and normalization settings. Training provenance remains in checkpoint metadata. Typing a run name alone is explicitly labeled as artifact selection, not recipe restoration. After job submission the UI adopts the API's canonical run name and normalized recipe.

Fresh training is rejected if the run already contains a checkpoint. Students must use a new name or deliberately enable continuation; continuation timesteps are an additional budget, not a lifetime target. PPO form submission is separate from the older free-text Duck Lab teaching workflow.

Complete reward and loss history

Previously, live reward history kept only the latest 240 samples and disappeared when the server state was lost. Studio now restores collection, update, and raw episode-return records from `training-metrics.jsonl`. It displays raw training returns, rollout rewards, policy loss, and value loss, with a full-data download.

For continued runs, the backend recovers prior stages using recorded checkpoint and sidecar hashes. This also handles a base checkpoint copied into a new run before continuation: the source path can refer to the new run, but the saved hashes identify the actual sibling ancestor. Filename similarity is not used as proof. Stage offsets are cumulative, but each stage remains a separate curve. BC/Dagger initialization is a separate non-PPO stage.

History is preserved before a journal can be truncated, archive failures stop the operation, and complete/cold-restart reads do not double-count a journal. Malformed or partial records are ignored without inventing values. A first measurement is labeled at its actual recorded step, never fabricated at zero.

How to read the graphs: normalized rollout reward can fall as the running normalizer changes, even while raw episodic returns improve. Episodic return also depends on survival duration. PPO policy loss and value loss need not be monotonic. None of these curves alone certifies a physical skill; use the independent episode-level acceptance tests.

Review and provenance

The evaluation panel now exposes episode counts, measurement ranges, exact thresholds, each episode's details, and the raw saved report. Switching to an unavailable run immediately removes the previous run's verdict and video. Manual review is bound to the selected run, evaluated source hash, and media identity, rather than carrying over to another policy.

Successful rendering creates `render/evidence.json`. It records hashes of the source policy, checkpoint sidecar when applicable, explicit Dance reference, MP4, and contact sheet, plus the environment-settings key. Inputs are checked before and after rendering. The UI allows visual approval only when this receipt matches the evaluated source and nominal rendering projection of its settings. Rendering disables randomization; it does not visualize every randomized evaluation domain. The evaluation itself retains its original settings and verdict. All UI ONNX download links honor the review gate, but raw local artifact APIs are analysis interfaces, not an authorization or hardware-safety boundary. A reference edited in place invalidates the old Dance verdict. Legacy/stale videos remain viewable but cannot satisfy the package gate. Hardware deployment remains disabled.

Four verified outcomes

Scenario	Selected final run	Fresh audit	What is demonstrated
Dance	dance-e2e-20260907-low-noise	5/5	Eight-second Bachata excerpt, upright tracking, positive dynamic and leg gain
Swing	swing-e2e-20260907-v3	5/5	24 seconds from rest, symmetric span about 162.32 degrees, tensioned strings and valid geometry
Running	running-e2e-20260907-v4	5/5	12-second running episodes, signed progress, bilateral support switches and aerial phases
Stilts	stilts-e2e-20260907-v3	5/5	10-second gait with the selected 2 cm stilt morphology, signed progress and bilateral stepping

The historical learning budgets are 4,001,792 fresh PPO transitions for Dance; 6,000,640 plus 2,097,152 for Running; 6,000,640 plus 1,048,576 for Stilts; and teacher BC/Dagger acquisition followed by 524,288 PPO transitions for Swing. Swing is not presented as pure PPO acquisition from scratch. Running trajectories can still curve, and the stilt result does not establish arbitrary-height ability.

All negative controls failed the strict skill tests. ONNX parity errors were at most approximately $1.43e-6$, below the audit threshold of $1e-4$. Fresh audit outputs and comparison videos were byte-identical to their earlier reference artifacts.

After the audits, all four API videos were also regenerated through Studio with the exact saved evaluation settings. Existing MP4s and contact sheets were backed up first. Every regenerated MP4 was byte-identical to its original and now has a matching render receipt. No trained policy bytes were changed.

Validation and reproduction

From duck-viewer/:

```
npm test
npm run lint
./node_modules/.bin/tsc --noEmit --incremental false
npm run build
npm run e2e:studio
```

The browser regression opens all four real saved runs, verifies full ancestor and current-stage history, plays each complete-duration MP4, and checks mobile layout at 390 pixels. It also tests immediate stale-evidence removal, review reset, clip/horizon form payloads, and adoption of the server-normalized recipe. The submission-payload tests are intercepted; they do not pretend to be training.

The actual selected-clip integration run uses the existing API driver:

```
node scripts/rlx-dance-api-e2e.mjs --execute \
  --run YOUR_NEW_SMOKE_RUN \
  --recipe-json ../rlx/artifacts/studio-ui-20260908/clip-selection-smoke.json \
  --report ../rlx/artifacts/studio-ui-20260908/your-smoke-report.json \
  --timeout-seconds 180
```

This completed train, evaluate, render, and export with the selected Cumbia reference. Its one-second MP4 and ONNX were generated successfully. The four learned-skill claims instead come from the separate full-duration policy audits.

The native RLX suite passed **418 tests, with 2 skipped**. Optional mjlab tests were excluded because that optional stack is not installed. No claim is made that every unrelated checkout or optional GPU suite passes; existing unrelated microduck_local suite failures were not changed by this UI task.

The updated Studio Node suite passed **110 tests**; ESLint, strict TypeScript, and the production Next.js build also passed. Tests include cold history reads, ancestor-hash recovery, failed archive preservation, stale policy/clip evidence, and changed render-source/media/settings rejection.

Evidence index

- Fresh policy audit and command manifest
- Native RLX test log
- Actual clip-selection API integration receipt
- Regenerated video provenance and backup hashes
- Desktop/mobile browser verification
- Dance reward/loss lifecycle
- Swing reward/loss lifecycle
- Running reward/loss lifecycle
- Stilt reward/loss lifecycle

The audit manifest describes the audit phase, when existing API videos were only revalidated. The later render-refresh receipt records their subsequent actual regeneration and matching media hashes. These are distinct verification stages.

Remaining boundaries

This is a local simulation prototyping workflow. It does not establish hardware safety, robustness to arbitrary domains, multiple independent training seeds, full-source dance imitation, or guaranteed learning for every selectable clip. The existing student book and its historical evidence are preserved. This report documents the newer UI and provenance behavior; earlier UI screenshots in the book remain historical rather than being silently relabeled as current.

Measured training evidence

These original-resolution plots were regenerated by the fresh audit from the saved training journals. They are measurements, not illustrative curves. For Running and Stilts, these audit plots describe the selected continuation; Studio's full lifecycle view additionally exposes the hash-matched base stage.

Dance: reward and PPO losses

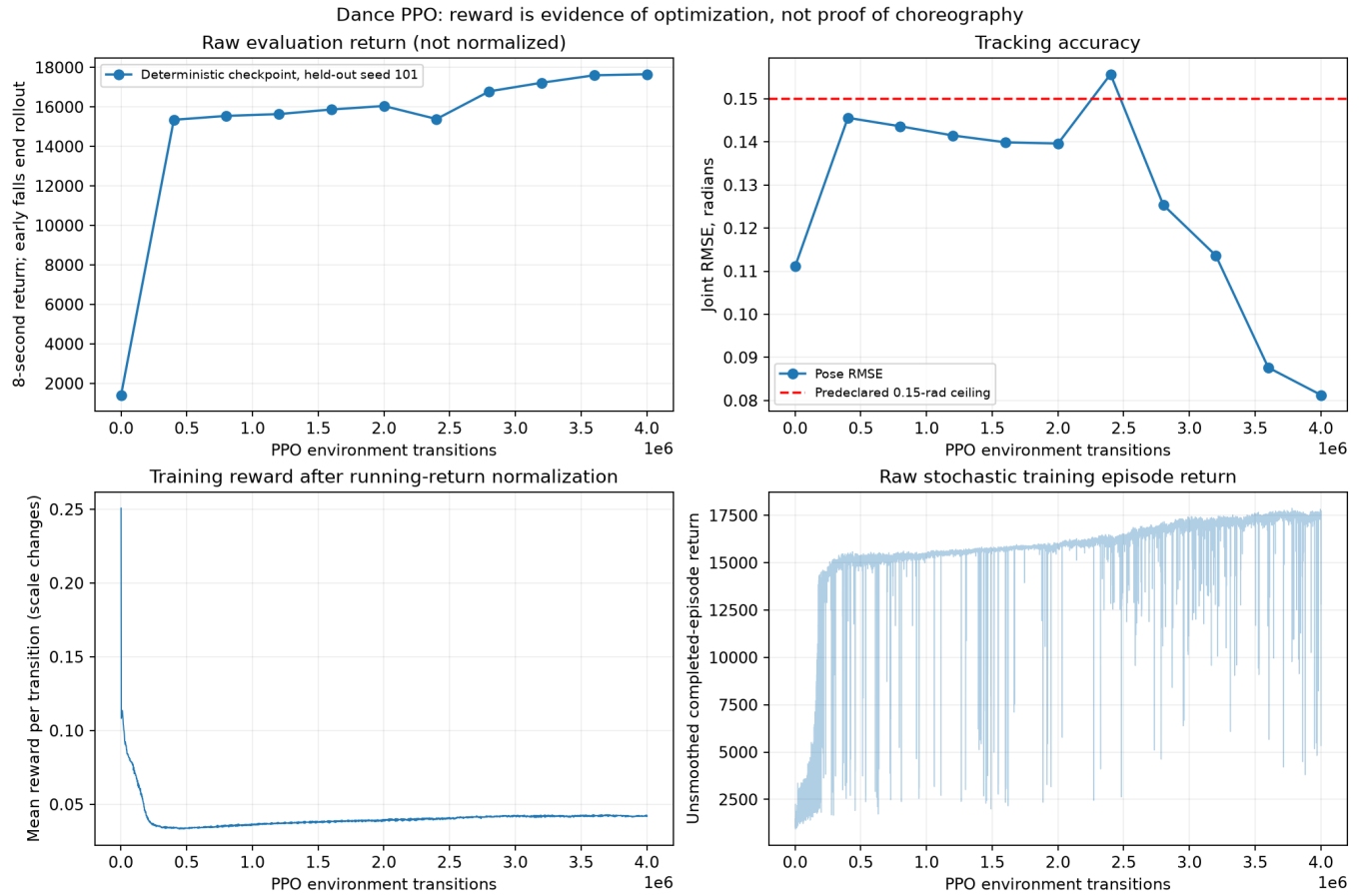


Figure 1: Dance measured reward and evaluation progression.

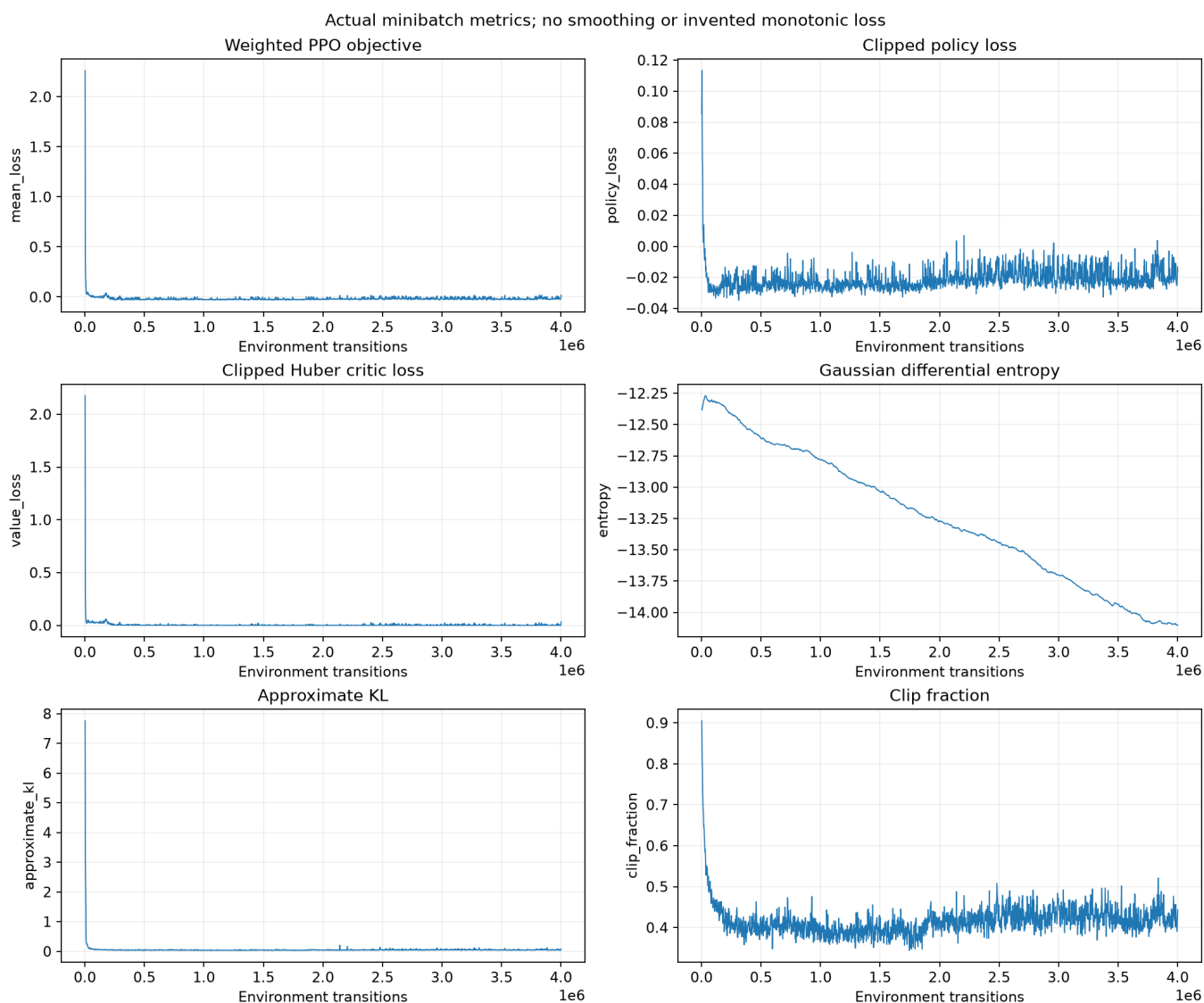


Figure 2: Dance PPO optimizer diagnostics.

Swing: reward and PPO losses

The initializer already acquired the swing through teacher training. These plots evaluate PPO refinement, not acquisition from an untrained Gaussian actor.

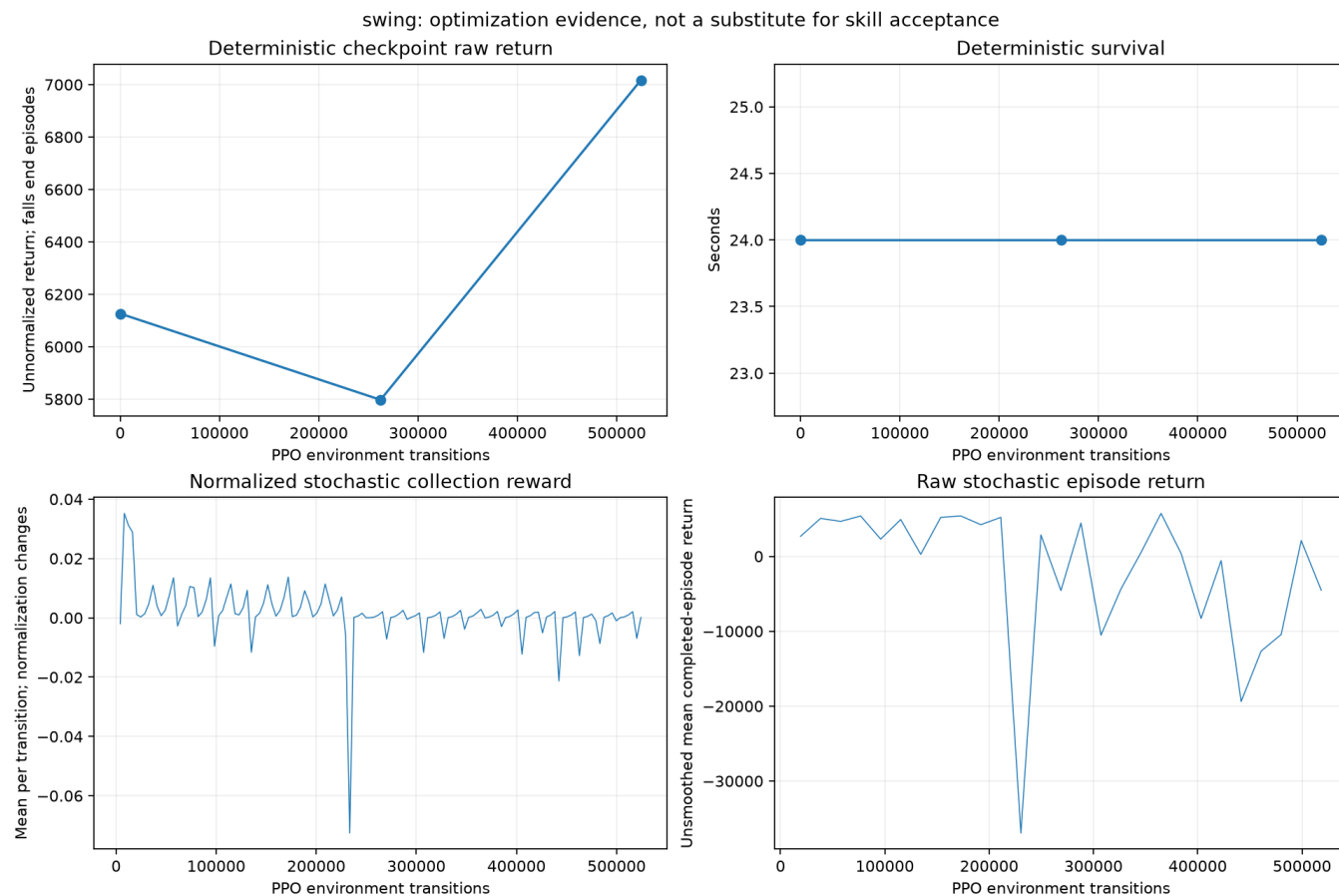


Figure 3: Swing measured reward and evaluation progression.

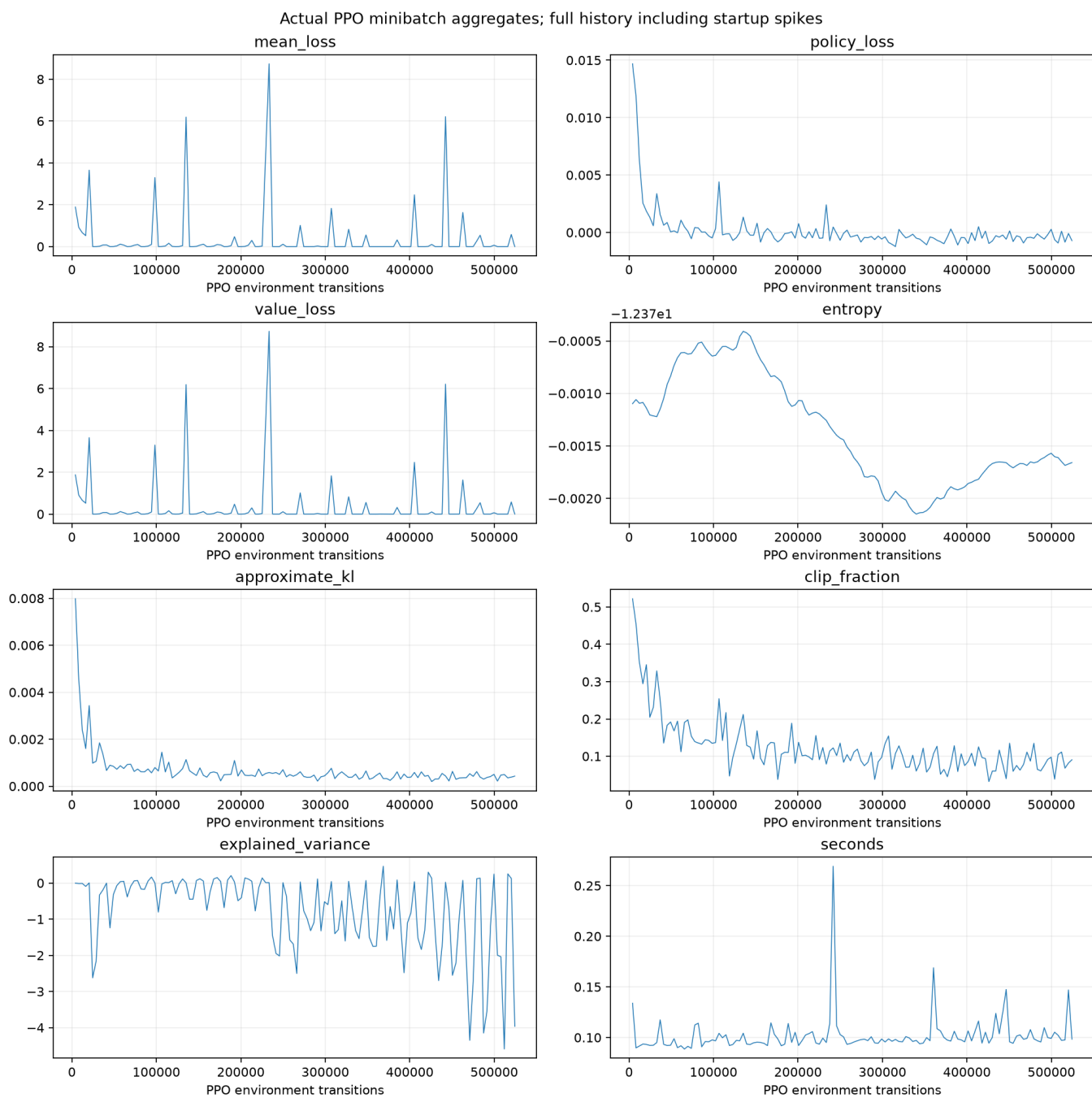


Figure 4: Swing PPO optimizer diagnostics. Negative critic explained variance remains a limitation.

Running: reward and PPO losses

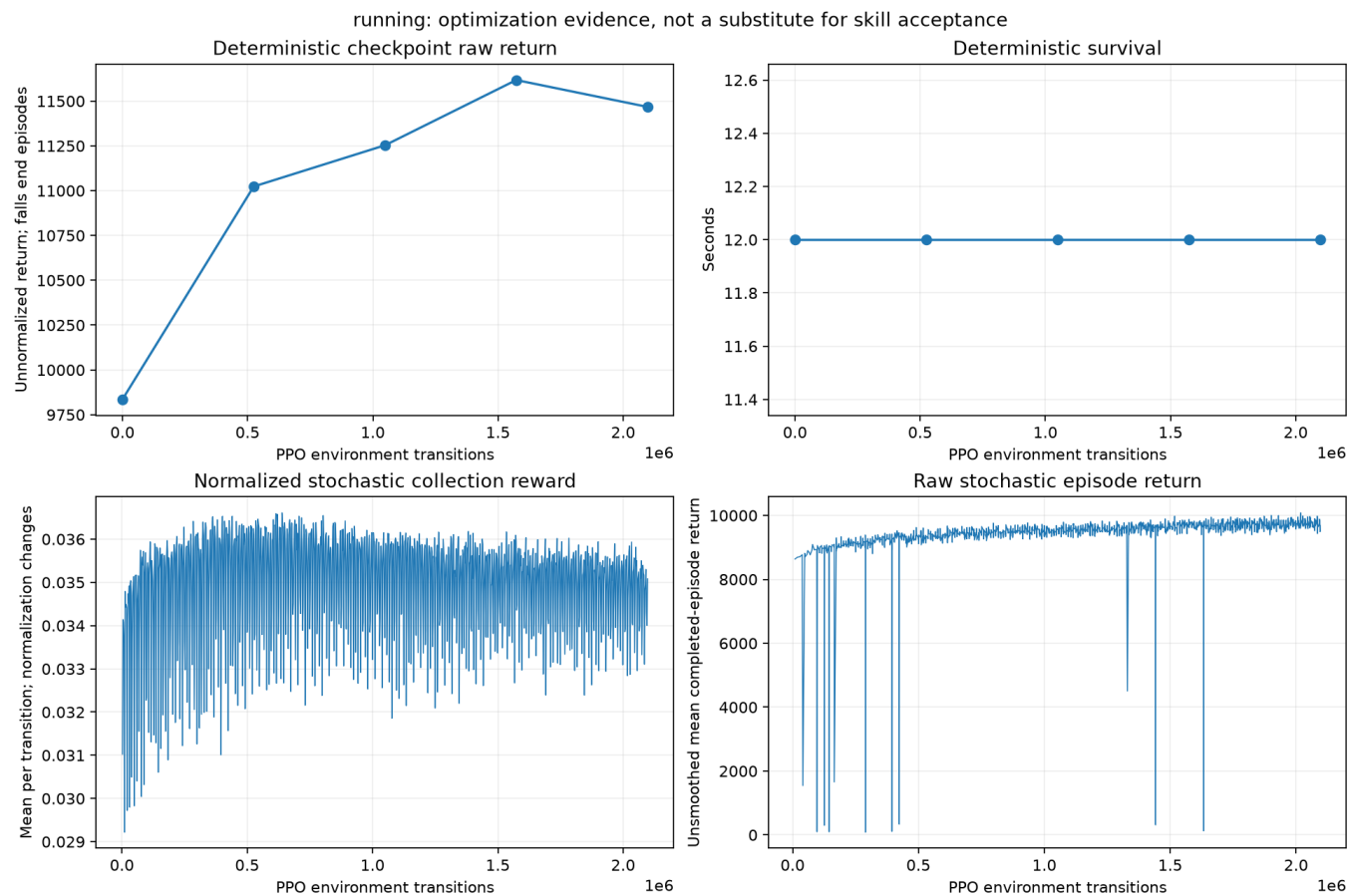


Figure 5: Running continuation reward and evaluation progression.

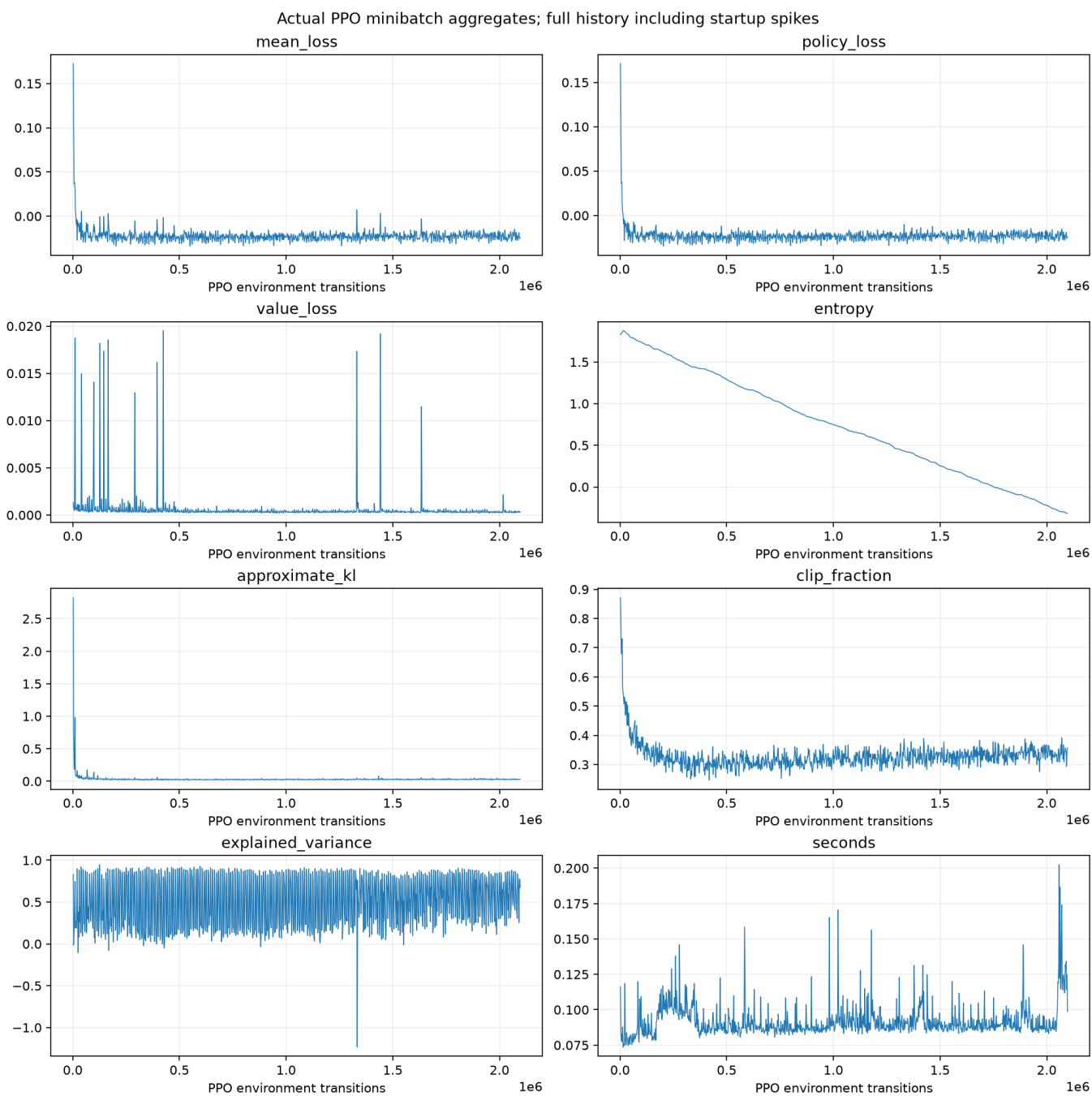


Figure 6: Running continuation PPO optimizer diagnostics.

Stilt walking: reward and PPO losses

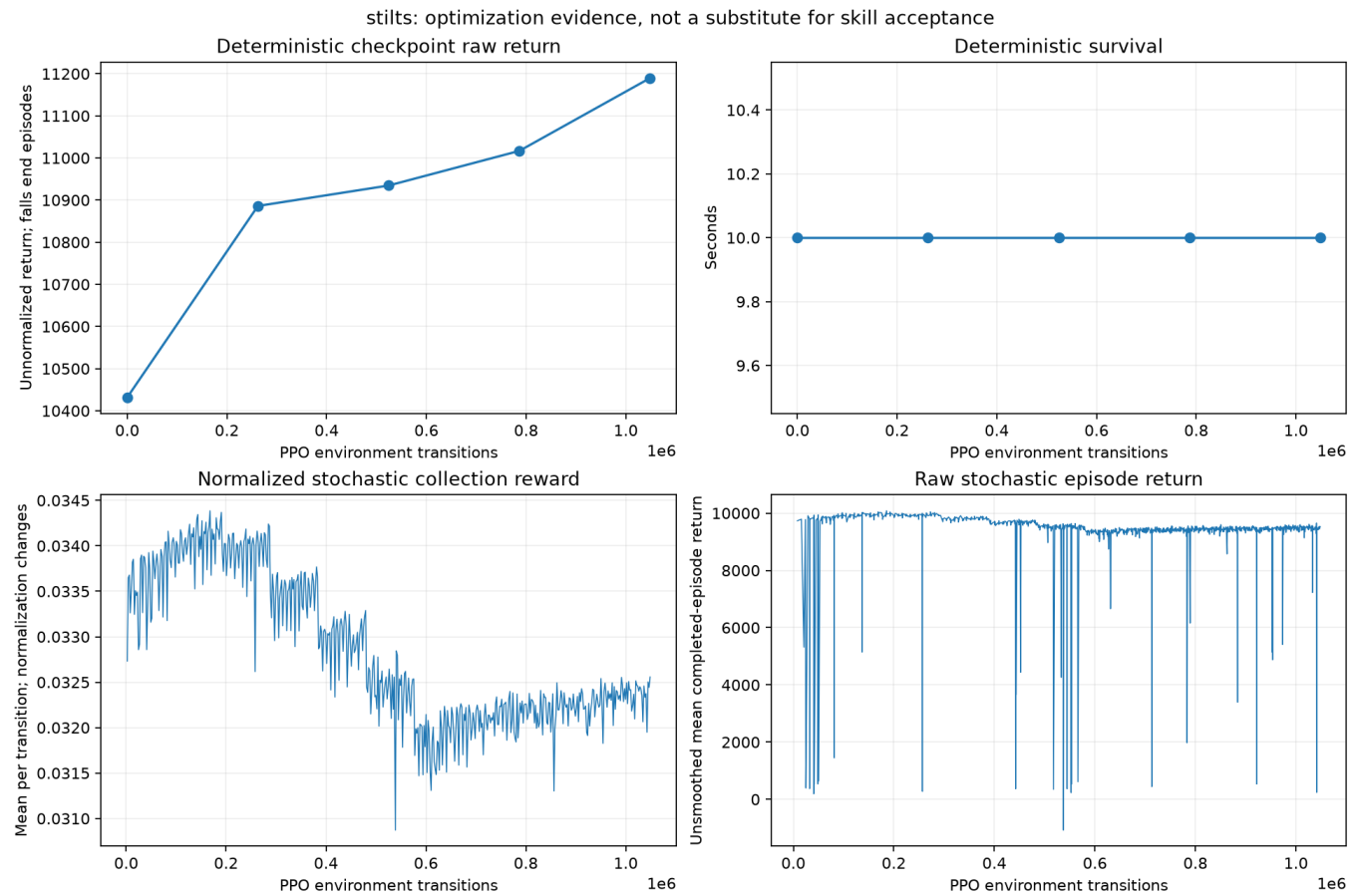


Figure 7: Stilt walking continuation reward and evaluation progression.

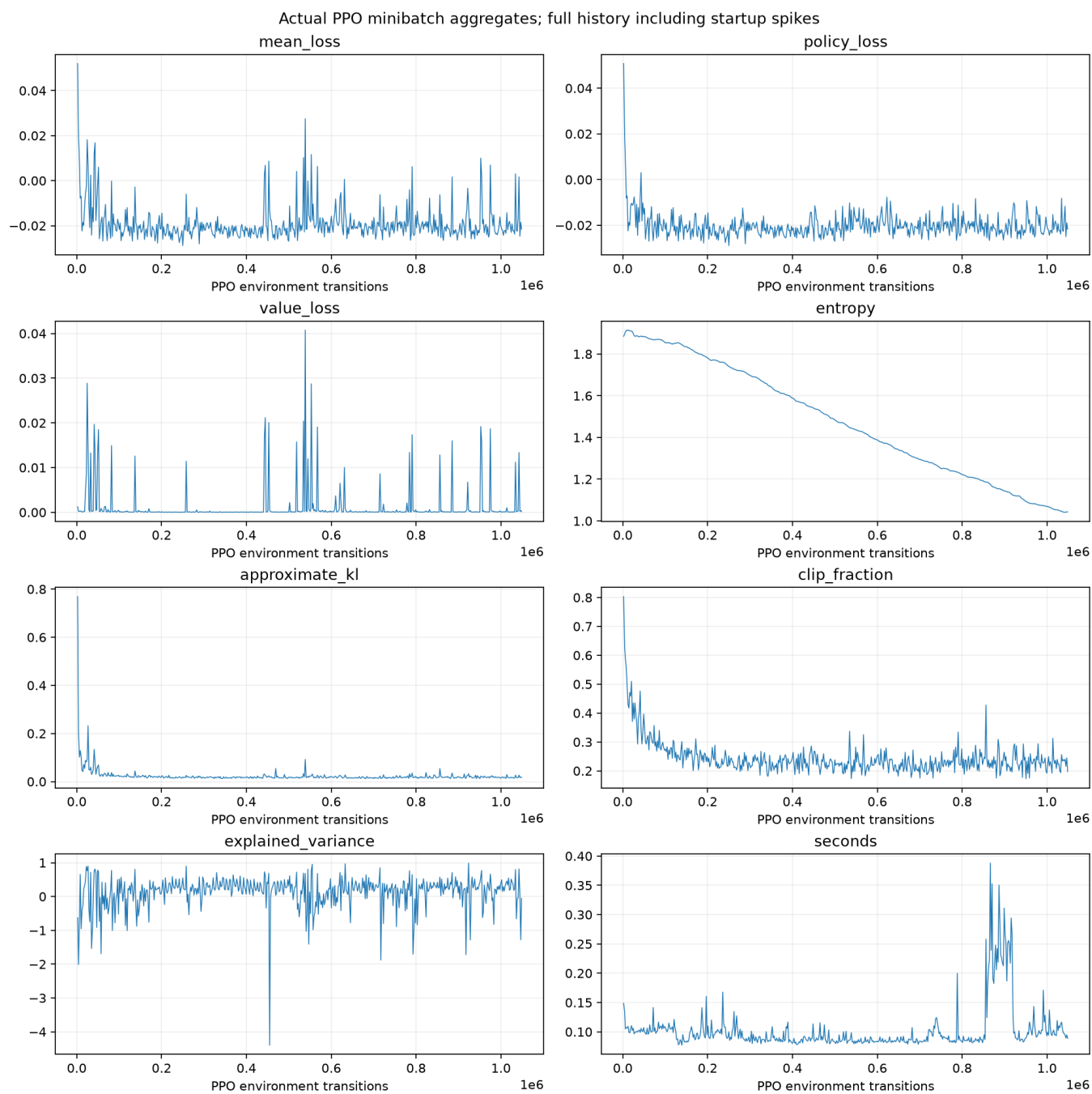


Figure 8: Stilt walking continuation PPO optimizer diagnostics.